SYSTEM DESIGN & ARCHITECTURE SPECIFICATION

Technical architecture, module boundaries and data flow

AI-Powered Multimodal Assistive Application for People with Mild Cognitive Impairment (MCI)

Mridula J B, Navaneeth S Narayan, Neeraj Ram B, Sonal Santhosh

Prof. Binu Rajan M R, Assistant Professor, Dept. of CSE, SCTCE

1. Architecture Goals

- Keep patient interaction simple while hiding technical complexity.
- Separate AI modules so they can be developed and tested independently.
- Keep sensitive processing/storage under controlled infrastructure where feasible.
- Use structured databases for deterministic information and vector retrieval for unstructured memory.
- Allow the system to continue core schedule/reminder functions even when a non-critical AI service fails.

2. High-Level Architecture

Recommended logical architecture:

Layer	Components	Responsibility
Presentation	Patient Mobile App; Caregiver Dashboard	User interaction, camera/microphone/location permissions, visual/voice output.
API/Application	Django/REST backend; Authentication; RBAC; Decision/Orchestration	Business rules, API endpoints, access control, coordination.
AI Services	Object Detection/Tracking; Face Recognition; STT; Embeddings; RAG; LLM; optional emotion analysis	Specialized inference and contextual assistance.
Data	Relational DB; Vector Store; Object/Face metadata; Logs	Persistent structured data, searchable memories and audit information.
Notification	Local/Push notification service	Reminders and caregiver alerts.

3. Logical Data Flow

The supplied project presentation depicts the mobile application capturing multimodal data and sending it to a Django backend. The backend routes data to OpenCV/computer vision, NLP/RAG and face-recognition components, then combines results through a decision engine before delivering patient assistance or caregiver dashboard information.

1. Patient action/input → Mobile app.
2. Mobile app → authenticated backend API.
3. Backend → appropriate AI/business service.
4. AI service → structured result or retrieval context.
5. Decision/orchestration layer → final action.
6. Patient receives assistance; caregiver receives monitoring/alert output where applicable.

4. Module Design

Module	Responsibility
MOD-01 Authentication & RBAC	Login, sessions/tokens, roles, permissions.
MOD-02 Patient/Caregiver Management	Profiles, relationships and access linkage.
MOD-03 Schedule & Calendar	Daily routines, appointments, recurring activities.
MOD-04 Reminder & Medication	Reminder generation, medication schedules and status.
MOD-05 Object Intelligence	Detection, tracking, scene/context analysis, observation storage.
MOD-06 Face Intelligence	Face detection, embedding, matching and identity context.
MOD-07 Conversational Memory	STT, cleaning/segmentation, embedding, vector retrieval.
MOD-08 AI Assistant	Intent/context routing, structured-data lookup, RAG, response generation.
MOD-09 Safety & Geofencing	Location monitoring, safe-zone evaluation and alerts.

MOD-10 Notifications	Patient reminders and caregiver alerts.
MOD-11 Well-being	Meditation/relaxation content and playback.
MOD-12 Activity/Audit Logging	Important user/system events and monitoring history.

5. Object Intelligence Design

Pipeline:

7. Camera input
8. Frame extraction
9. Object detection
10. Object tracking
11. Scene/context understanding
12. Object observation storage
13. Patient query
14. Last-observation retrieval
15. Text/voice response

Suggested stored observation fields:

Field	Purpose
object_id	Identify object category/instance where supported.
timestamp	When object was observed.
location/context	Room, scene label or available approximate location.
confidence	Detection confidence.
source	Camera/session that generated the observation.

6. Face Recognition Design

16. Camera input → face detection → face embedding → registered-person matching → identity/relationship cue → optional RAG context enrichment

The system must distinguish recognition from identification confidence. Unknown/low-confidence faces shall not be force-mapped to a registered person.

7. Conversational RAG Design

Recommended pipeline:

17. Microphone/text input
18. Speech-to-text if required
19. Conversation cleaning/segmentation
20. Context extraction (person/topic/time where available)
21. Embedding generation
22. Vector search
23. Metadata filtering/reranking
24. Prompt/context construction
25. LLM response generation
26. Text/voice response

8. Hybrid Retrieval Strategy

Use deterministic retrieval for structured data and semantic retrieval for memory. For example, 'What medicine do I take at 8 AM?' should query medication/schedule tables. 'What did my daughter tell me last Sunday?' should use conversational memory retrieval, optionally filtered using the recognized person and time context.

9. Geofencing Design

27. Caregiver defines a named safe zone with center and radius.
28. Location updates are received from the mobile device when permission/service is available.
29. Backend or device-side safety service evaluates the point against the configured boundary.
30. On violation, an alert record is created.
31. Notification service delivers the alert to the caregiver.
32. System logs the event.

10. Database Relationships

Relationship	Description
Caregiver → Patient	Caregiver manages authorized patient.
Patient → Medication	Patient has medication records.
Medication → MedicationSchedule	Medication can have one or more scheduled times.
Patient → Appointment	Patient has calendar appointments.
Patient → Reminder	Patient has reminder rules/events.
Patient → Person	Patient has familiar-person profiles.
Person → FaceEmbedding	Person can have recognition representations.
Patient → ObjectObservation	Object observations belong to patient context.
Patient → Conversation	Patient's supported conversation records.
Conversation → MemoryEmbedding	Conversation/event can be vectorized for retrieval.
Patient → Geofence	Patient can have configured safe zones.
Patient → ActivityLog	System records relevant activity.

11. API Boundary Recommendations

Endpoint Group	Example Operations
Auth	POST /auth/login, POST /auth/refresh, POST /auth/logout
Patient	GET /patients/me, GET /patients/{id}
Schedule	GET/POST/PATCH /schedule
Appointments	GET/POST/PATCH /appointments
Medications	GET/POST/PATCH /medications
Reminders	GET/POST/PATCH /reminders
Objects	POST /objects/detect, GET /objects/{name}/last-seen
People	GET/POST/PATCH /people
Face	POST /face/recognize
Memory	POST /memory/ingest, POST /memory/search
Assistant	POST /assistant/query
Geofence	GET/POST/PATCH /geofences
Location	POST /location/events
Alerts	GET /alerts, POST /alerts/{id}/ack
Activity	GET /activity

Exact routes may change during implementation; the important requirement is to keep module boundaries stable and document the final API.

12. Failure Isolation

Failure	Core System Behavior
Object AI down	Schedules, reminders and caregiver management continue.
Face AI down	Identity cue unavailable; other assistant functions continue.
RAG/LLM down	Structured schedule/medication queries continue; assistant reports limited capability.
GPS unavailable	Geofence status becomes unavailable/limited; system records condition.
Notification provider down	Pending notification is persisted/retried where supported.

13. Recommended Development Order

33. Backend project, authentication and RBAC.
34. Patient/caregiver data model.
35. Schedule, medication, appointments and reminders.
36. Patient and caregiver UI.
37. Object detection and observation storage.
38. Face recognition and familiar-person management.
39. Vector store and RAG ingestion/retrieval.
40. AI assistant orchestration.
41. Geofencing and alerts.
42. Voice/TTS and well-being module.
43. Integration testing, security testing and performance benchmarking.